Transaction Processing

Database II - Lecture 6

1. Introduction

Managing Data is the critical role in each and every organization. To achieve the hike in business they need to manage data efficiently. DBMS provides a better environment to store and retrieve the data in an economical and efficient manner. User can store and retrieve data through various sets of instructions. These sets of instructions do several read and write operations on database. These processes are denoted by a special term "Transaction" in DBMS.

Transaction is the execution of user program in DBMS. It is different from the execution of the program external to DBMS. In other words it can be stated as the various read and write operations done by the user program on the DBMS, when it is executed in DBMS environment.

Transaction Management plays a crucial role in DBMS. It is responsible for the efficiency and consistency of the DBMS. Partial transaction let the database in an inconsistency state, so they should be avoided.

2. Key Notations in Transaction Management

The key notations in transaction management are as follows:

Object: The smallest Data item which is read or updated by the Transaction is called as Object in this case.

Transaction: Transaction is represented by the symbol T. It is termed as the execution of query in DBMS.

Read Operation: Read operation on particular object is notated by symbol R (object-name).

Write Operation: Write operation on particular object is notated by symbol W (object-name).

Commit:. This term used to denote the successful completion of one Transaction. *Abort:* This term used to denote the unsuccessful interrupted Transaction.

3. Concept of Transaction Management

User program executed by DBMS may claim several transactions. In the web environment, there is a possibility to several users' attempt to access the data stored in same database. To maintain the accuracy and the consistency of the database several scheduling algorithms are used. To improve the effective throughput of the DBMS we need to enforce certain concurrent executions in DBMS. Transaction Manager is responsible for scheduling the Transactions and providing the safest path to complete the task. To maintain the data in the phase of concurrent access and system failure, DBMS need to ensure four important properties. These properties are called as ACID properties.

4. ACID Properties of DBMS

ACID is an acronym for Atomicity, Consistency, Isolation, and Durability.

A – Atomicity

C – Consistency

I – Isolation
D – Durability
Atomicity and Durability are closely related.
Consistency and Isolation are closely related.
The ACID properties are explained as follows.

4.1 Atomicity and Durability

Atomicity

Either all Transactions are carried out or none are. The meaning is the transaction cannot be subdivided, and hence, it must be processed in its entirety or not at all. Users should not have to worry about the effect of incomplete Transactions in case of any system crash occurs. Transactions can be incomplete for three kinds of reasons:

1. Transaction can be aborted, or terminated unsuccessfully. This happens due to some anomalies arises during execution. If a transaction is aborted by the DBMS for some internal reason, it is automatically restarted and executed as new.
2. Due to system crash. This may be happen due to Power Supply failure while one or more Transactions in execution.
3. Due to unexpected situations. This may be happen due to unexpected data value or be unable to access some disk. So the transaction will decide to abort. (Terminate itself).

Durability

If the System crashes before the changes made by a completed Transaction are written to disk, then it should be remembered and restored during the system restart phase.

Durable means the changes are permanent. Once a transaction is committed, no subsequent failure of the database can reverse the effect of the transaction.

Partial Transaction

If the Transaction is interrupted in the middle way it leaves the database in the inconsistency state. These types of transactions are called as Partial Transactions.

Partial Transactions should be avoided to gain consistency of database. To undo the operations done by the Partial Transactions DBMS maintains certain log files. Each and every moment of disk writes are recorded in this log files before they are reflected to disk. These are used to undo the operations done when the system failure occurs.

4.2 Consistency and Isolation

Consistency

Users are responsible for ensuring transaction consistency. User who submits the transaction should make sure the transaction will leave the database in a consistent state.

Example

If the transaction of money between two accounts “A” and “B” is manually done by the user, then first thing he has to do is, he deducts the amount (say $100) from the account “A” and add it with the account “B.” DBMS do not know whether the user subtracted the exact amount from account “B.”

User has to do it correctly. If the user subtracted $99 from account “B” instead of $100 DBMS is not

responsible for that. This will leave DBMS in inconsistency state.

Isolation

In DBMS system, there are many transaction may be executed simultaneously. These transactions should be isolated to each other. One's execution should not affect the execution of the other transactions. To enforce this concept DBMS has to maintain certain scheduling algorithms. One of the scheduling algorithms used is Serial Scheduling.

5. Serial Scheduling

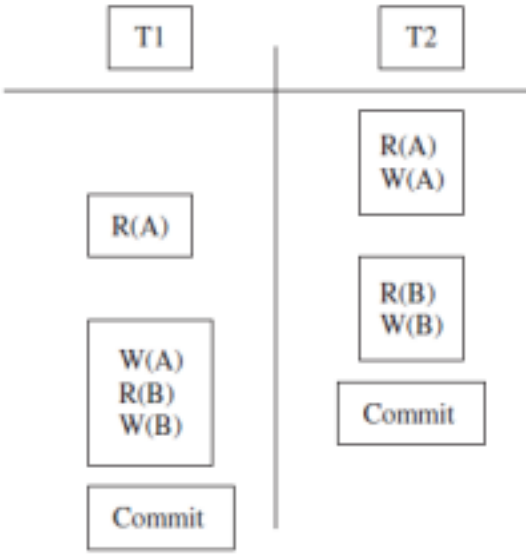
In this scheduling method, transactions are executed one by one from the start to finish. An important technique used in this serial scheduling is interleaved execution.

Interleaved Execution

In DBMS to enforce concurrent Transactions, several Transactions are ordered in a serial manner and executed on by one according to the schedule. So there will be the switching over of execution between the Transactions. This is called as Interleaved Execution.

Example

The example for the serial schedule is illustrated in Figure below:



Serial scheduling with Interleaved Execution

Explanation

In the above example, two Transactions T1, T2 and two Objects A, B are taken into account.

Commit denotes successful completion of both Transactions.

First one read and one write operation are done on the object A by Transaction T2. This is followed by T1. It does one write operation on object A. The same procedure followed by others for further. Finally, both Transactions are ended successfully.

6. Anomalies due to Interleaved Transactions

If all the transactions in DBMS systems are doing read operation on the Database then no problem will arise. When the read and write operations done alternatively there is a possibility of some type of anomalies. These are classified into three categories.

1. Write–Read Conflicts (WR Conflicts)
2. Read–Write Conflicts (RW Conflicts)

3. Write–Write Conflicts (WW Conflicts)

6.1 WR Conflicts

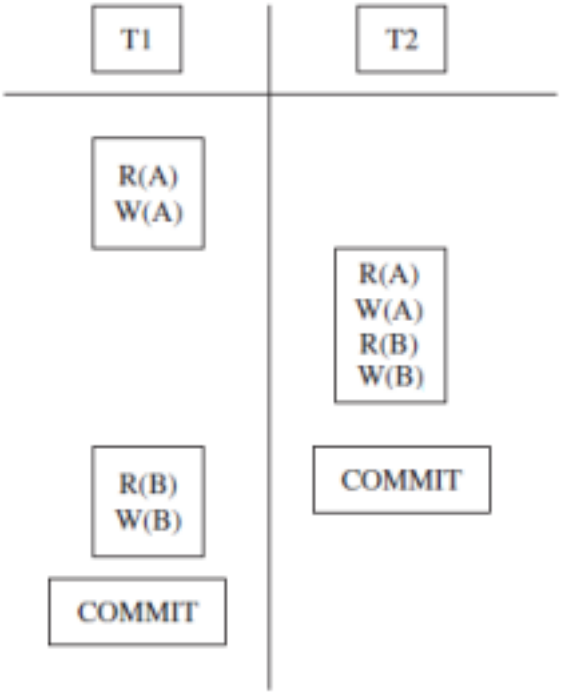
Dirty Read

This happens when the Transaction T2 is trying to read the object A that has been modified by another Transaction T1, which has not yet completed (committed). This type read is called as dirty read.

Example

Consider two Transactions T1 and T2, each of which, run alone, preserves database consistency. T1 transfers $100 from A to B, and T2 increments both A and B by 6%.

If the Transaction is scheduled as illustrated below.



Reading uncommitted data

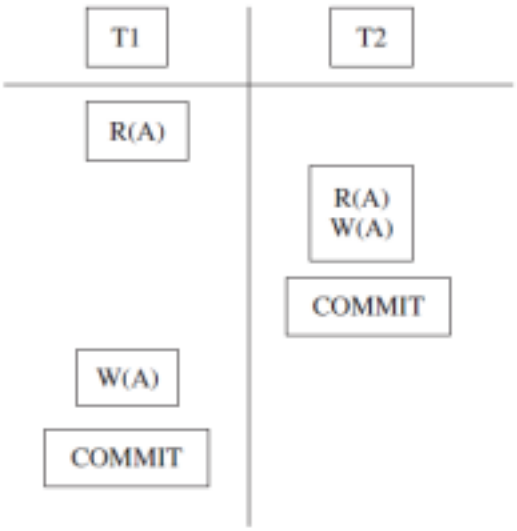
Explanation

Suppose if the transactions are interleaved according to the above schedule then the account transfer program T1 deducts $100 from account A, then the interest deposit program T2 reads the current values of accounts A and B and adds 6% interest to each, and then the account transfer program credits $100 to account B. The outcome of this execution will be different from the normal execution like if the two instructions are executed one by one. This type of anomalies leaves the database in inconsistency state.

6.2 RW Conflicts

Unrepeatable Read

In this case anomalous behavior could result is that a Transaction T2 could change the value of an object A that has been read by a Transaction T1, while T2 is still in progress. If T1 tries to read A again it will get different results. This type of read is called as Unrepeatable Read. **Example**
If "A" denotes an account. Consider two Transactions T1 and T2. Duty of T1 and T2 are reducing account A by $100. Consider the following Serial Schedule as shown:



RW conflicts

Explanation

At first, T1 checks whether the money in account A is more than $100. Immediately it is interleaved and T2 also checks the account for money and reduce it by $100. After T2, T1 is tried to reduce the same account A. If the initial amount in A is $101 then, after the execution of T2 only $1 will remain in account A. Now T1 will try to reduce it by $100. This makes the Database inconsistent.

6.3 WW Conflicts

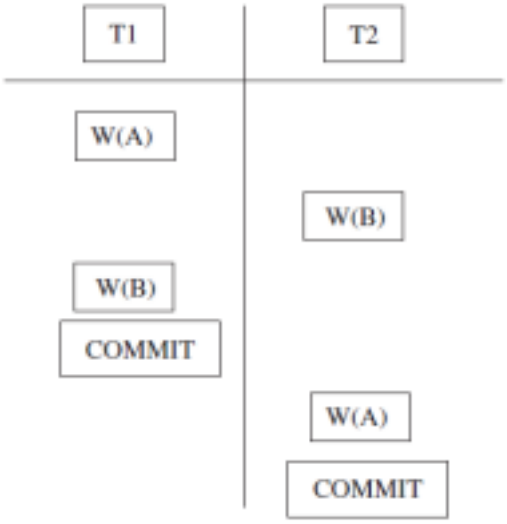
The third type of anomalous behavior is that one Transaction is updating an object while another one is also in progress.

Example

Consider the two Transactions T1, T2.

Consider the two objects A, B.

Consider the following Serial Schedule as illustrated in Figure below:



WW conflicts

Explanation

If A and B are two accounts and their values have to be kept equal always Transaction T1 updates both objects to 3,000 and T2 updates both objects to 2,000. At first T1 updates the value of object A to 3,000. Immediately T2 makes A as 2,000 and B as 2,000 and committed After the completion of T2, T1 updates B to 3,000. Now the value of A is and value of B is 3,000, they are not equal. Constraint violated in this case due to serial scheduling

7. Lock-Based Concurrency Control

Concurrency Control is the control on the Database and Transactions which are executed concurrently to ensure that each Transaction completed healthy. Concurrency control is concerned with preventing loss of data integrity due to interference between users in a multiuser environment.

Need for Concurrency Control

In database management system several transactions are executed simultaneously. In order to achieve concurrent transactions, the technique of interleaved execution is used. But in this technique there is a possibility for the occurrence of certain anomalies, due to the overriding of one transaction on the particular Database Object which is already referred by another Transaction.

Lock-Based Concurrency Control

It is the best method to control the concurrent access to the Database Objects by providing suitable permissions to the Transactions. Also it is the only method which takes less cost of Time and less program complexity in terms of code development.

Key Terms in Lock-Based Concurrency Control

Database Object: Database Object is the small data element, the value of which one is altered during the execution of transactions.

***Lock*:** Lock is a small object associated with Database Object which gives the information about the type of operations allowed on a particular Database Object.

Lock can be termed as a type of permission provided by the transaction manager to the

transactions to do a particular operation on a Database Object.
The transaction must get this permission from Transaction Manager to access any Database Object for alteration. Locking mechanisms are the most common type of concurrency control mechanism. With locking, any data that is retrieved by a user for updating must be locked, or denied to other users, until the update is complete.

Locking Protocol

It is the set of rules to be followed by each transaction, to ensure that the net effect of execution of each Transaction in interleaved fashion will be same as, the result obtained when the Transactions executed in serial fashion.

Strict Two-Phase Locking (Strict 2PL)

It is a most widely used locking protocol. It provides few rules to the Transactions to access the Database Objects. They are:

Rule 1:

If a Transaction "T" wants to read, modify an object, it first requests a shared, exclusive lock on the Database Object respectively.

Rule 2:

All Locks held by the Transaction will be released when it is completed.

Shared Lock. It is type of lock established on a Database Object. It is like a component which is sharable within all active transactions. A Database Object can be shared locked by more than one number of transactions. To get a shared lock on particular Database Object the Database Object should satisfy the following condition.
Condition: It should not be exclusively locked by any of the other Transactions.

Example

If a person updates his bank account then the Database will lock that Database Object exclusively to avoid RW conflicts. So the Transactions which are requesting to read that Database Object will be suspended until the completion of updating.

Exclusive Lock. It is type of lock established on a Database Object. It is like a component which cannot be shared within all active Transactions. It is dedicated to particular transaction; only that particular transaction can access and modify that object.
Condition. It should not be exclusively locked by any one of the other Transactions.

Example

Assume the situation in reservation of Bus tickets in KPN Travels agencies. Assume number of tickets remain in bus no. 664 AV is only one. Two persons who are in different places are trying to reserve that particular ticket. See the situation here that only one of them should be allowed to access the Database while the other should wait until previous one is completed. Otherwise one terminal will check the number of seats available and due to interleaved actions next terminal will do

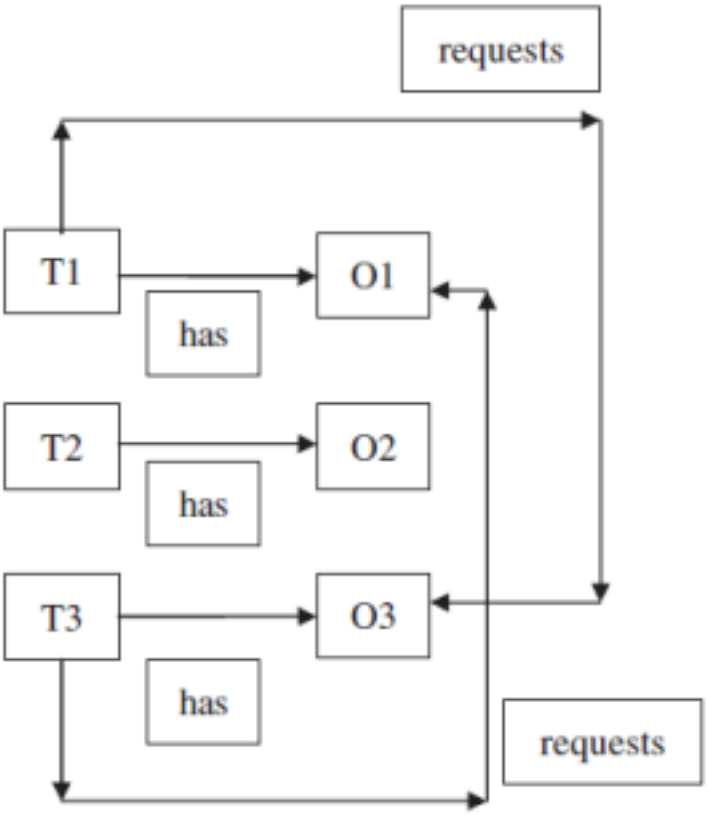
the same and finally both of them will try to modify the Database (Decrement the seats available) this leads to more anomalies in Database and finally Database will be left into inconsistent state.

Deadlock

Deadlock occurs within the Transactions in DBMS system. Due to this neither one will be committed. This is the dead end to the execution of transactions. DBMS has to use suitable recovery systems to overcome Deadlocks. Reasons for Deadlock Occurrence. Deadlock occurs mainly due to the Lock-Based Concurrency Control. The exclusive lock type will isolate one particular Database Object from the access of other transactions. This will suspend all the transactions who request Shared lock on that particular Database Object until the transaction which holds Exclusive lock on that object is completed. This will create a loop in Database which leads to Deadlock within transactions. This will leave the Database in inconsistent state.

Example

Assume, Transactions T1, T2, T3 as illustrated in Figure below Database Objects are O1, O2, O3.



Deadlock

Explanation

Here we can observe that the loop occurs between T1 and T3. Neither T1 nor T3 are completed.

Methods to Overcome Deadlock

Mostly it is not possible to avoid the occurrence of Deadlock. Most of the methods available are detection and recovery based.

- Detection from the Average Waiting Time of the Transaction

- Deadlock Detection algorithm

Recovery Mechanism

Once if Deadlock is found then we have several methods to release locked Transactions.

Method 1*:* Release of Objects from Big Transaction
Method 2: Restarting the Locked Transactions